Raven Malware Triage Platform

Detailed 7-Week Work Plan

Project summary

A static malware triage platform. A file is analyzed without being executed, and the system produces an evidence-backed report, a MITRE ATT&CK mapping, a containment playbook, and an automated response. The plan moves from data collection in week 1 to the self-healing capability in week 7.

Timeline at a glance

Week	Focus	Outcome
1	**Data collection**	Collect everything the project needs: reference material, safe test files, ATT&CK knowledge, and a safe lab setup.
2	**Design and architecture**	Turn the requirements into a clear technical design before writing code.
3	**Static analysis engine**	Build the backend that reads a file and extracts raw evidence.
4	**Detection and scoring**	Turn raw evidence into a transparent risk score.
5	**User interface**	Build the dashboard and connect it to the backend.
6	**ATT&CK and AI**	Add the ATT&CK matrix, the Claude investigation, and the response playbook.
7	**Self-healing**	Add self-healing, then test, document and present.

Week 1: Data collection

Goal: Collect everything the project needs: reference material, safe test files, ATT&CK knowledge, and a safe lab setup.

Tasks

- **Study the reference project.** Read the original Raven write-up: the problem (raw tool output that analysts must connect by hand), the pipeline, the tools used, and the challenges (context management, validating on unknown hashes).
- **Collect interface references.** Gather screenshots of the original dashboard: top bar, sidebar, analysis pipeline, agent hierarchy, sample overview, analyst chat.
- **Build the safe test set.** Clean notes file; text files containing ransom-note and dropper-style commands (inert); hand-built synthetic PE files with injection, packing and keylogging imports and no runnable code; the EICAR test file; real Windows executables copied from System32 (for example certutil and mshta) to check for over-flagging.
- **Prepare for real samples.** Review public sources such as MalwareBazaar and the handling rules: isolated virtual machine, network off, shared folders and clipboard off, snapshot before use, never double-click a sample.
- **Collect ATT&CK techniques.** List the techniques tied to the behaviors to detect (process injection, persistence, credential access, ransomware impact, defense evasion, command and control) with their IDs and tactics.
- **Write requirements.** Functional: upload, analysis, score, report, ATT&CK view, response. Non-functional: never execute a sample, explainable scoring, 32 MB limit, results understandable to analysts and managers.

Tools	Python, Windows tools, VirtualBox or VMware (optional), MITRE ATT&CK website
Deliverables	Test file set, requirements list, ATT&CK technique list, lab setup notes
Done when	Every test file has a documented purpose and expected result; requirements are written and agreed.

Week 2: Design and architecture

Goal: Turn the requirements into a clear technical design before writing code.

Tasks

- **Choose the stack and justify it.** Next.js and React (interface), FastAPI (API), pefile and YARA (analysis), Claude Agent SDK (investigation), Docker (optional isolation).
- **Design the pipeline.** Five stages: Ingest, Classify, Analyze, Reason, Report, and which component runs in each.
- **Design the agent hierarchy.** Orchestrator, static analyst, deobfuscation, scenario builder, parallel monitors (network, filesystem, registry, threat intel), adversarial critic, report writer, response planner. Decide which are real and which are displayed only in the first version.
- **Design the report format.** File info and hashes, score and verdict, findings with points and evidence, ATT&CK mapping, indicators, PE structure, playbook.
- **Define the API.** `POST /api/analyze` (file, options), `GET /api/report/{id}`, later `/api/health`.
- **Wireframe the interface** from the week 1 screenshots.
- **Plan the security model.** Samples are read as bytes only, stored read-only, size-limited, and isolated where possible.

Tools	Diagram tool, Figma or paper wireframes
Deliverables	Architecture diagram, wireframes, API specification, report schema
Done when	Each requirement maps to a component; the design can be explained in five minutes.

Week 3: Static analysis engine

Goal: Build the backend that reads a file and extracts raw evidence.

Tasks

- **Set up FastAPI** with an upload endpoint and a 32 MB size limit.
- **Hashing:** SHA-256 and SHA-1 of the file.
- **Entropy:** Shannon entropy for the whole file and for each PE section.
- **String extraction:** ASCII and UTF-16 strings, capped to keep memory and report size under control.
- **PE parsing with pefile:** machine type, compile timestamp, sections (name, entropy, writable and executable flag), imports and DLL names.
- **Indicator extraction:** regular expressions for URLs, IP addresses (validated, local addresses removed), domains, registry keys, file paths, emails.
- **Test** each component on the week 1 files and compare with expected results.

Tools	Python, FastAPI, pefile, regular expressions
Deliverables	Backend returning raw findings as JSON for any uploaded file
Done when	All test files analyze without errors; hashes match an independent tool; PE imports match the fixtures.

Week 4: Detection and scoring

Goal: Turn raw evidence into a transparent risk score.

Tasks

- **Write YARA rules:** encoded PowerShell, ransomware indicators, process injection APIs, UPX packing, Defender tampering.
- **Build the rule table.** Each rule has an ID, name, ATT&CK techniques, the imports and strings it looks for, the points it adds, and the minimum number of matching signs.
- **Add structural rules:** packed or encrypted sections, writable and executable sections, hardcoded network indicators, YARA match points.
- **Implement the score** as the sum of rule points capped at 100, with verdicts: 70 or more likely malicious, 40 to 69 suspicious, 15 to 39 low risk, below 15 no strong indicators. Every point is tied to visible evidence.
- **Tune against clean files.** Run real Windows executables and the clean notes file, and adjust points to reduce false positives.

Tools	YARA, Python
Deliverables	Scored findings with evidence, verdict logic, YARA rule file
Done when	Clean files score low; each synthetic fixture triggers the expected rules.

Week 5: User interface

Goal: Build the dashboard and connect it to the backend.

Tasks

- **Create the Next.js app** with the top navigation and the sidebar (investigation views and analysis tools).
- **Overview page:** upload box, Run and Clear buttons, five-step pipeline with running and complete states, agent hierarchy with a completed counter, sample overview panel, analyst chat panel.
- **Evidence page:** findings table, PE sections, indicators, strings sample.
- **Response Plan and Architecture pages.**
- **Connect to the backend** through a proxy rewrite, and animate the pipeline against the real request.
- **End-to-end testing** with every test file; fix environment and dependency problems.

Tools	Next.js, React, CSS, Node.js
Deliverables	Working interface that uploads a file and displays the real analysis
Done when	A user can load a file, run analysis, and read all pages without errors.

Week 6: ATT&CK and AI

Goal: Add the ATT&CK matrix, the Claude investigation, and the response playbook.

Tasks

- **Build the ATT&CK matrix:** a mapping of the techniques used by the rules to names and tactics, grouped by tactic, with a link to each technique page.
- **Add the Claude Agent SDK investigator.** It works in a throwaway folder on a read-only copy of the sample with Read, Grep, Glob and Bash, a turn limit, and instructions never to execute the sample. It looks for what static analysis missed.
- **Generate the containment playbook** from the extracted indicators and findings (isolate host, block indicators, check persistence, credential reset).
- **Write the executive summary** in plain language.
- **Full testing** with all fixtures, then bug fixing and cleanup.

Tools	Claude Agent SDK, Anthropic API key, MITRE ATT&CK data
Deliverables	ATT&CK tab, optional Claude investigation, playbook and summary
Done when	Each finding shows the right technique; the agent returns evidence-backed notes or fails gracefully.

Week 7: Self-healing

Goal: Add self-healing, then test, document and present.

Tasks

- **Platform self-healing:** stage retries with backoff; results marked ok, recovered or degraded; analysis continues if YARA or the agent fails; a `/api/health` endpoint; the interface retries up to three times if the backend is unreachable.
- **Host response (automatic):** when the score reaches 70, quarantine the file (read-only, with a record of its origin) and block malicious public IPs found as URL hosts in the Windows firewall, using validated inputs with no shell commands.
- **Safety controls:** score threshold, audit log of every action, undo (restore file, remove firewall rule), and a dry-run list of suggested commands for manual steps.
- **Watch folder mode:** new files in a chosen folder are analyzed and, if malicious, quarantined automatically.
- **Final testing** with every fixture, including failure cases (backend stopped, invalid input, low score, no administrator rights).
- **Documentation and presentation:** README, user guide, final report, and a demo script.

Tools	FastAPI, Python, Windows firewall, Next.js
Deliverables	Working self-healing system, final report, presentation and demo
Done when	The demo shows a malicious file detected, quarantined and restored with undo; the platform recovers from a simulated failure.

Risks and mitigations

Risk	Mitigation
Automatic quarantine of a harmless file (false positive)	Score threshold of 70, audit log, one-click undo, and tuning against clean files in week 4.
Antivirus deletes test files	Keep test files in a dedicated folder; allow the antivirus to scan them; regenerate if removed. Do not add exclusions on the main machine.
Handling real malware	Only inside an isolated virtual machine with the network off and a snapshot taken first.
Dependency or build problems (Node, Python, YARA)	Test the setup early in week 3 and week 5; keep a Docker option as a fallback.
No API key for the Claude investigation	The investigation is optional; the platform falls back to static analysis only.
Time overrun	Reserve the final week for testing and documentation; reduce the scope of automatic actions first.